Problem Set 3

Algorithms Design - SCE

Due: June 4, 2026

General Instructions

Answer concisely and clearly. Submission in pairs is mandatory. For every problem submit:

- A clear DP formulation (state, recurrence, base cases).
- Pseudo-code or code.
- A concise proof of correctness.
- Time and space analysis.

Problem 1

You are given a non-negative array $cost[0..n-1]$. You may start on stair 0 or 1 and move either $+1$ or $+2$ steps each turn. Compute the minimum total cost to step beyond stair $n-1$.

1. Present an $O(n)$-time, $O(1)$-extra-space bottom-up DP.
2. Prove correctness and analyse its running time and memory usage.

Problem 2

Given $1 \le m \le 300$ balloons with values $1 \le nums[i] \le 100$, bursting balloon i earns:

$$nums[L] \cdot nums[i] \cdot nums[R]$$

where L and R are the nearest intact balloons to the left/right (use 1 if none). Return the maximum total coins obtainable.

1. Define an interval DP state $dp(l, r)$ for the open interval (l, r) and derive its recurrence.
2. Give pseudo-code running in $O(m^3)$ time.
3. Prove correctness and analyse time/space complexity.

Problem 3

An $m \times n$ grid $dungeon[i][j]$ contains positive (healing) or negative (damage) integers. A knight starts at (0,0), moves only right or down, and must arrive at $(m-1, n-1)$ with health > 0. Find the minimum initial health required.

1. Explain why a backwards DP is natural here; define $dp[i][j]$.
2. Derive the recurrence and base cases.
3. Prove your algorithm is correct and runs in $O(mn)$ time and $O(n)$ extra space.

Problem 4

You are given an array $prices[0..n-1]$ of daily stock prices. You may buy and sell the stock multiple times but must obey:

- You may hold at most one share at a time.
- After you sell a share, you must wait exactly one day (the "cool-down") before buying again.

Maximise your total profit.

1. Formulate a DP with constant-sized state describing "holding", "just sold", and "resting" situations, yielding $O(n)$ time and $O(1)$ extra space.
2. Prove correctness and provide a tight complexity bound.

Problem 5

Given arrays $startTime$, $endTime$, and $profit$ for non-pre-emptive jobs, choose a subset of non-overlapping jobs maximising total profit.

1. Design an $O(n \log n)$-time DP based on sorting by end time and binary searching the next compatible job.
2. Prove correctness and analyse its complexity.
3. **Extension.** Suppose each job has difficulty d_i and a total budget B. Briefly outline how to incorporate the remaining budget into your DP state and discuss the new complexity.